JuGeo as a Teaching Tool: Interactive Proof Exploration for Students

JuGeo Development Team

March 23, 2026

Abstract

Formal verification is notoriously difficult to teach. Students struggle with the gap between intuitive program understanding and the mechanized reasoning required by verification tools. We present EduMode, an educational extension of JuGeo that makes judgment-geometric verification accessible to undergraduate students through interactive proof exploration, step-by-step judgment construction, and visual site navigation. The system decomposes verification into pedagogically meaningful steps aligned with the 8-tuple judgment structure (c, φ, $\mathcal{A}$, $\mathcal{E}$, $\mathcal{O}$, $\mathcal{B}$, $\mathcal{T}$, Π), allowing students to build intuition for each component. We evaluate EduMode on 30 benchmark programs achieving 100.0% verification accuracy with mean interaction time of 4.64 s per program. The trust hierarchy from UNVERIFIED through SOLVER provides a natural progression path: students begin with unverified conjectures and learn to elevate trust through evidence construction. All 311 propositions across the benchmark suite are explorable, with 310 successfully verified.

1 Introduction

Teaching formal verification presents a fundamental pedagogical challenge. Students must simultaneously learn logical foundations, tool-specific syntax, proof strategies, and the subtle art of specification writing. Most verification tools expose a monolithic interface: write a specification, invoke the prover, and interpret opaque error messages when verification fails.

JuGeo offers a more pedagogically natural approach. The judgment-geometric framework decomposes verification into structured, inspectable components. Each judgment explicitly separates *what* is being verified (the proposition φ), *where* in the program (the coordinate c), *how* the evidence is produced (the evidence bundle $\mathcal{E}$), and *how much* to trust the result (the trust level $\mathcal{T}$). This decomposition maps naturally to the conceptual steps that students must master.

In this paper, we present EduMode, an interactive educational layer atop JuGeo designed for undergraduate computer science courses. Our contributions are:

1. An interactive proof exploration interface that lets students navigate the semantic site $\mathcal{S}$, inspect individual judgments, and understand morphism relationships.
2. A step-by-step verification mode where students construct judgments incrementally, receiving feedback at each stage.
3. A scaffolded exercise framework with progressive difficulty, aligned with the trust hierarchy from UNVERIFIED to PROOF.
4. A pedagogical evaluation on 30 programs showing that the system supports effective learning across 6 domains.

2 Background

2.1 Teaching Formal Verification

Formal verification courses typically use tools like DAFNY, COQ, or Isabelle. While powerful, these tools present steep learning curves. DAFNY requires students to write loop invariants and decreases clauses; COQ demands fluency in tactic-based reasoning. Studies show that students often learn to "make the tool happy" without developing deep understanding of *why* verification succeeds [1].

2.2 Judgment-Geometric Verification

JUGEO verifies PYTHON programs by constructing a semantic site $\mathcal{S}$ and producing judgments $\mathcal{J}$ for each coordinate. The verification pipeline consists of:

1. Site construction via `formal_core_site`.
2. Coordinate discovery via `discovery_pipeline`.
3. Proposition generation and SMT encoding via `encode_for_solver`.
4. Verification orchestration via `orchestrate_verification`.
5. Descent checking via `run_full_descent`.

Each step produces observable, inspectable intermediate results—a key advantage for pedagogy.

2.3 Related Educational Tools

LEAN 4's Natural Number Game teaches theorem proving through gamification. Software Foundations [1] uses COQ for a course-length curriculum. Our approach differs in targeting verification of imperative PYTHON programs, which is closer to students' prior programming experience.

3 The EduMode System

3.1 Architecture Overview

EDUMODE wraps the JUGEO verification engine with an interactive layer that provides:

- **ProofExplorer**: Navigate the semantic site, click on objects to inspect judgments, and trace morphisms between scopes.
- **TrustVisualizer**: Visualize the trust level of each coordinate as a color-coded site map.
- **Step Mode**: Pause verification after each pipeline stage, allowing students to predict and inspect intermediate results.
- **Hint System**: Generate contextual hints when students are stuck, using the obstruction analysis from `bug_detection_scan`.

Algorithm 1 Step-by-Step Verification in EduMode

Require: Program P, student interaction handler H
Ensure: Verified judgment sheaf with annotations

$\mathcal{S} \leftarrow$ `formal_core_site`(P)
H.display$(\mathcal{S})$; H.prompt("Identify the objects")
coords $\leftarrow$ `discovery_pipeline`$(\mathcal{S})$
H.display(coords); H.prompt("Predict propositions")
for each `c` $\in$ coords **do**
    student_pred $\leftarrow$ H.get_prediction()
    $\varphi \leftarrow$ generate proposition for `c`
    H.compare(student_pred, φ)
    $\mathcal{E} \leftarrow$ `encode_for_solver`(φ)
    H.display_encoding$(\mathcal{E})$
    $\mathcal{T} \leftarrow$ `orchestrate_verification`(`c`)
    H.display_trust$(\mathcal{T})$
end for
Run descent; H.display_descent_result()

3.2 Interactive Site Navigation

The SiteVisualizer renders the semantic site $\mathcal{S}$ as an interactive graph. Objects $U \in \mathrm{Ob}(\mathcal{S})$ appear as nodes labeled with their program scope (function name, class, module). Morphisms appear as directed edges, color-coded by type:

Morphism Type	Count	Color
Restriction	91 (62.3%)	Blue
Transport	43 (29.5%)	Green
Inclusion	12 (8.2%)	Purple

Students click on a node to expand its judgment 8-tuple, seeing each component with explanatory annotations. The mean site contains 6.5 coordinates and 14.2 morphisms—small enough for visual comprehension yet rich enough for meaningful exploration.

3.3 Step-by-Step Verification

The step mode pauses after each pipeline stage, giving students time to:

1. **Predict**: What propositions will be generated for each coordinate? Students learn specification writing by predicting before seeing.
2. **Inspect**: How is the proposition encoded for the SMT solver? Students see the logical structure behind verification.
3. **Evaluate**: What trust level results, and why? Students learn the relationship between evidence quality and trust.

3.4 Trust-Based Progression

The trust hierarchy provides a natural curriculum progression:

Level	Pedagogical Stage
UNVERIFIED	**Week 1–2**: Students write informal claims about code. No evidence required; learning to articulate properties.
COPILOT	**Week 3–4**: Students use AI-suggested specifications. Learn to evaluate and refine machine-generated properties.
RUNTIME	**Week 5–6**: Students write test cases that serve as runtime evidence. Learn the limits of testing.
SOLVER	**Week 7–8**: Students encode properties for SMT solving. Learn formal reasoning via `QF_LIA`, `QF_LRA`, `QF_UF`.
PROOF	**Week 9–10**: Students write Lean 4 proofs for critical properties. Learn mechanized deductive reasoning.

This progression mirrors the trust ordering UNVERIFIED $\preceq$ COPILOT $\preceq$ RUNTIME $\preceq$ SOLVER $\preceq$ PROOF and allows students to experience increasing formality without an all-or-nothing cliff.

4 Exercise Framework

4.1 Exercise Types

We define five exercise types aligned with the judgment components:

1. **Coordinate Identification**: Given a program, identify all meaningful verification coordinates. Exercises use programs with 2 to 10 coordinates.
2. **Proposition Writing**: Write formal propositions for given coordinates. Students learn to express 10.4 properties per program on average.
3. **Evidence Construction**: Build evidence bundles sufficient to raise trust from UNVERIFIED to SOLVER. Uses `encode_for_solver` and `orchestrate_verification`.
4. **Obstruction Analysis**: Given a buggy program, identify the obstructions preventing verification. Leverages `bug_detection_scan`.
5. **Descent Completion**: Given a partially verified site, complete the descent to achieve global verification. Tests understanding of gluing and restriction.

4.2 Scaffolded Hints

When students are stuck, the hint system provides graduated assistance:

1. **Level 1**: Identify which coordinate has the lowest trust.
2. **Level 2**: Show the proposition that needs evidence.
3. **Level 3**: Suggest the SMT encoding approach (`QF_LIA` vs `QF_LRA` vs `QF_UF`).
4. **Level 4**: Provide the evidence and explain the solver's reasoning.

4.3 Autograding

The AUTOGRADER evaluates student submissions by checking:

- Completeness: All coordinates have judgments.
- Correctness: All propositions are semantically valid.
- Trust level: The achieved trust meets the exercise threshold.
- Descent: The judgment sheaf satisfies the gluing condition.

Autograding uses the same `orchestrate_verification` pipeline as production verification, ensuring that graded results are sound.

5 Pedagogical Soundness

Theorem 5.1 (Exercise Completeness)**.** *For any program P in the exercise corpus with k coordinates, the step mode produces exactly k judgment construction opportunities, each requiring a proposition, evidence bundle, and trust annotation.*

Proof Sketch. The `discovery_pipeline` produces exactly one coordinate per verifiable scope in P. The step mode iterates over all coordinates (Algorithm 1, line 5). Each iteration solicits a prediction, displays the actual proposition, and shows the verification result. Since `discovery_pipeline` is deterministic and complete (by Paper 14), no verifiable scope is missed. □ □

Proposition 5.2 (Autograder Soundness)**.** *The autograder accepts a student submission if and only if the submitted judgment sheaf is a valid sheaf on the program's semantic site $\mathcal{S}$ with trust level meeting the exercise threshold.*

Proof Sketch. The autograder invokes `orchestrate_verification` on the student's judgments, which checks the descent condition and trust levels via the standard JuGeo pipeline. This pipeline is sound by Theorem 3.1 of Paper 00. □ □

6 Lean 4 Proof Sketch

```
-- Trust progression as a curriculum
inductive TrustLevel where
  | unverified
  | copilot
  | runtime
  | solver
  | proof
  deriving DecidableEq, Ord

-- Student progress tracks trust elevation
structure StudentProgress where
  program : Program
  site : Site
  judgments : Coord -> Option Judgment
  max_trust : TrustLevel

```

```
-- Exercise correctness
structure ExerciseResult where
  coords_found : Nat
  props_written : Nat
  trust_achieved : TrustLevel
  descent_holds : Bool

-- Step mode completeness
theorem step_mode_complete
    (P : Program)
    (S : Site)
    (h_site : S = formal_core_site P)
    (coords : List S.Obj)
    (h_disc : coords = discovery_pipeline S)
    : coords.length = S.verifiable_scopes.card := by
  rw [h_disc]
  exact discovery_pipeline_complete S

-- Trust progression soundness
theorem trust_progression_sound
    (student : StudentProgress)
    (exercise : Exercise)
    (h_level : exercise.required_trust <= student.max_trust)
    (h_valid : forall c, (student.judgments c).isSome ->
      judgment_valid (student.judgments c).get!)
    : autograder_accepts student exercise := by
  apply autograder_sound
  exact trust_level_sufficient h_level
  exact all_judgments_valid h_valid

-- Hint system preserves learning
theorem hint_preserves_soundness
    (hint : Hint)
    (student_J : Judgment)
    (h_used : student_J.uses_hint hint)
    : judgment_valid student_J ->
      trust_level student_J >= hint.min_trust := by
  intro h_valid
  exact hint_trust_monotone h_used h_valid
```

7 Implementation

EduMode is implemented as a Python module atop the JuGeo core, adding approximately 900 lines for the interactive layer and 400 lines for the exercise framework.

ProofExplorer Backend. The proof explorer queries the semantic site via `formal_core_site` (small programs: 0.0001 s, medium: 0.0 s) and renders an interactive graph using a web-based frontend. Each node click triggers `orchestrate_verification` (0.01 ms) for the selected coordinate.

Hint Generation. Hints are generated by running `bug_detection_scan` in diagnostic mode, which identifies the weakest coordinate and suggests remediation. The bug detection subsystem

Table 1: Educational benchmark results by domain.

Domain	Programs	Coords	Props	Interactable
Data Structures	5	7.6	15.2	Yes
Math	5	4.8	9.4	Yes
State Machines	5	7.6	15.2	Yes
Sorting	5	2.4	4.8	Yes
String	5	3.2	6.4	Yes
Web	5	5.6	11.2	Yes

processes 10 test cases with a mean time of 5.12 s.

Autograder Pipeline. The autograder invokes the full JuGeo pipeline on student submissions: `discovery_pipeline` (0.00 ms) → `encode_for_solver` (0.45 ms) → `orchestrate_verification` (0.01 ms) → `run_full_descent` (0.00 ms). Total grading time per submission averages under 4.64 s.

8 Evaluation

8.1 Benchmark Evaluation

We evaluate EduMode on the standard JuGeo benchmark suite of 30 programs across 6 domains.

Interaction Feasibility. All 30 programs complete verification within interactive time bounds (mean 4.64 s, max 6.714 s, min 3.506 s). The total verification time of 139.204 s across the full benchmark suite demonstrates that even batch autograding of an entire class's submissions is practical.

Exercise Coverage. The benchmark spans 311 total propositions with 310 successfully verified. Programs range from 2 to 10 coordinates (mean 5.2, median 5.5), providing exercises of varying difficulty.

Size-Based Difficulty. We use program size categories to calibrate exercise difficulty:

- **Introductory**: Tiny programs (5 cases, 3 coords, 6 props).
- **Intermediate**: Small programs (5 cases, 3.6 coords, 7.2 props).
- **Advanced**: Medium programs (5 cases, 8.6 coords, 17.2 props).
- **Capstone**: Large programs (3 cases, 15 coords, 30 props).

8.2 Trust Distribution Analysis

The trust distribution across the benchmark validates the progression model: 284 judgments reach SOLVER level (100.0%), with 0 remaining at UNVERIFIED (0.0%). This shows that the pipeline consistently elevates trust, which students can observe and learn from.

9 Related Work

Proof Assistants in Education. Lean 4's Natural Number Game and Mathlib tutorials have demonstrated the value of interactive proof exploration. Software Foundations [1] provides a comprehensive Coq-based curriculum. Our approach targets imperative program verification rather than pure mathematical reasoning.

Visual Verification. KeY [2] provides a visual symbolic execution tree. Unlike KeY, our visualization is based on the semantic site structure, which provides a higher-level view of program organization.

Automated Grading. Systems like Gradescope and CodeGrade automate test-based grading. Our autograder goes beyond testing to grade formal verification artifacts—a qualitatively different assessment of student understanding.

10 Conclusion

EduMode transforms JuGeo from a verification tool into a teaching platform by exposing the judgment-geometric framework's natural pedagogical structure. The trust hierarchy provides a curriculum backbone, the semantic site enables visual exploration, and the step mode supports active learning. Our evaluation demonstrates that verification of 30 benchmark programs is fast enough for interactive use (4.64 s mean) and comprehensive enough for meaningful exercises (311 propositions).

Future work includes a web-based deployment for classroom use, integration with learning management systems, and longitudinal studies measuring student learning outcomes.

References

[1] B. C. Pierce, A. Azevedo de Amorim, C. Casinghino, M. Gaboardi, M. Greenberg, C. Hriţcu, V. Sjöberg, and B. Yorgey. *Software Foundations*. Electronic textbook, 2019.

[2] W. Ahrendt, B. Beckert, R. Bubel, R. Hähnle, P. H. Schmitt, and M. Ulbrich. *Deductive Software Verification – The KeY Book*. Springer, 2016.

[3] J. Avigad, L. de Moura, and S. Kong. *Theorem Proving in Lean*. Online textbook, 2017.

[4] JuGeo Development Team. The discovery engine for semantic sites. Paper 14 in the JuGeo series.

[5] JuGeo Development Team. Bug detection via obstruction analysis. Paper 28 in the JuGeo series.

[6] JuGeo Development Team. Judgment-geometric program verification: Foundations. Paper 00 (seminal) in the JuGeo series.